

Java Backend Architecture

Interview Series

Chapter 15.9

Microservices Testing Strategies

50+ Interview Q&A

Code Examples

Architecture Diagrams

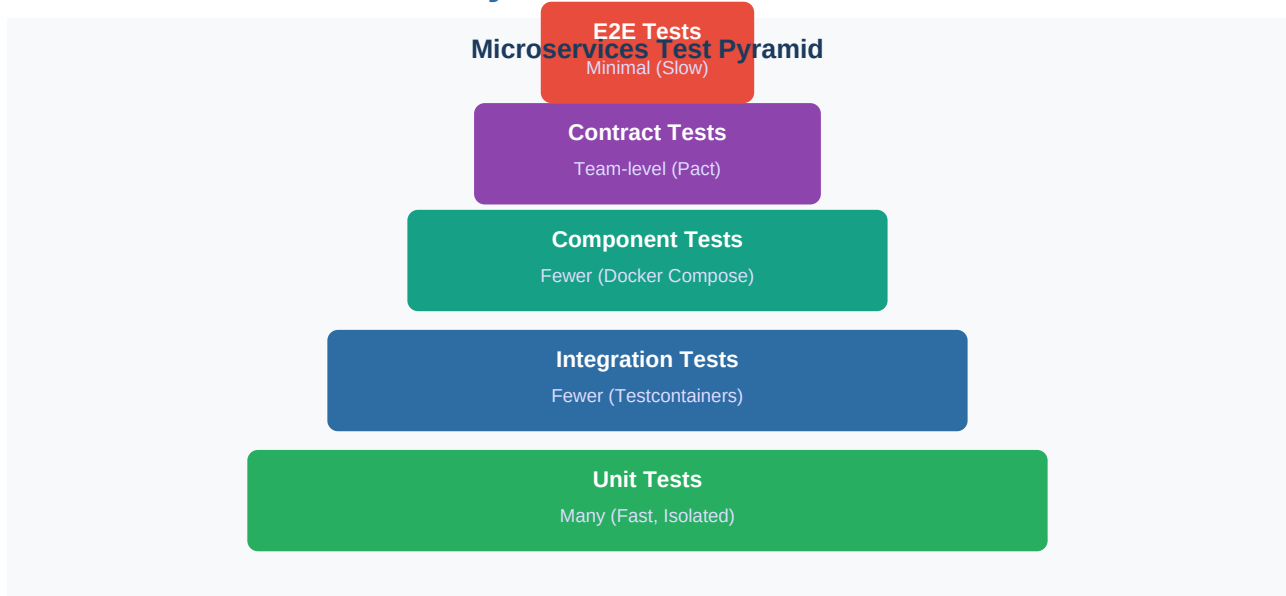
Advanced Topics

Chapter 15.9 – Microservices Testing Strategies

Overview

Testing microservices requires a layered strategy. The test pyramid guides investment: many fast unit tests, fewer integration tests with real infrastructure via Testcontainers, component tests verifying service behavior end-to-end, Pact contract tests ensuring inter-service compatibility, and minimal E2E tests for critical paths only.

The Microservices Test Pyramid



Unit Tests (Base of Pyramid)

- Mock ALL external dependencies (database, HTTP clients, message queues)
- Test business logic in isolation — fastest feedback loop
- JUnit 5 + Mockito: `@ExtendWith(MockitoExtension.class)`, `@Mock`, `@InjectMocks`
- Test edge cases, error paths, and boundary conditions

```
@ExtendWith(MockitoExtension.class) class OrderServiceTest { @Mock UserClient userClient;
@Mock OrderRepository repo; @InjectMocks OrderService service; @Test void
placeOrder_userNotFound_throws() { when(userClient.getUser(any())).thenReturn(new
UserNotFoundException()); assertThrows(OrderException.class, () ->
service.placeOrder(orderDto)); } }
```

Integration Tests (Testcontainers)

- Spin up real dependencies (PostgreSQL, Redis, Kafka) in Docker containers
- `@Testcontainers` + `@Container` annotations for lifecycle management
- Tests a single service with real infrastructure, no other services
- `@DynamicPropertySource` injects container connection URLs into Spring context

```
@SpringBootTest @Testcontainers class UserRepositoryIntegrationTest { @Container static
PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15");
```

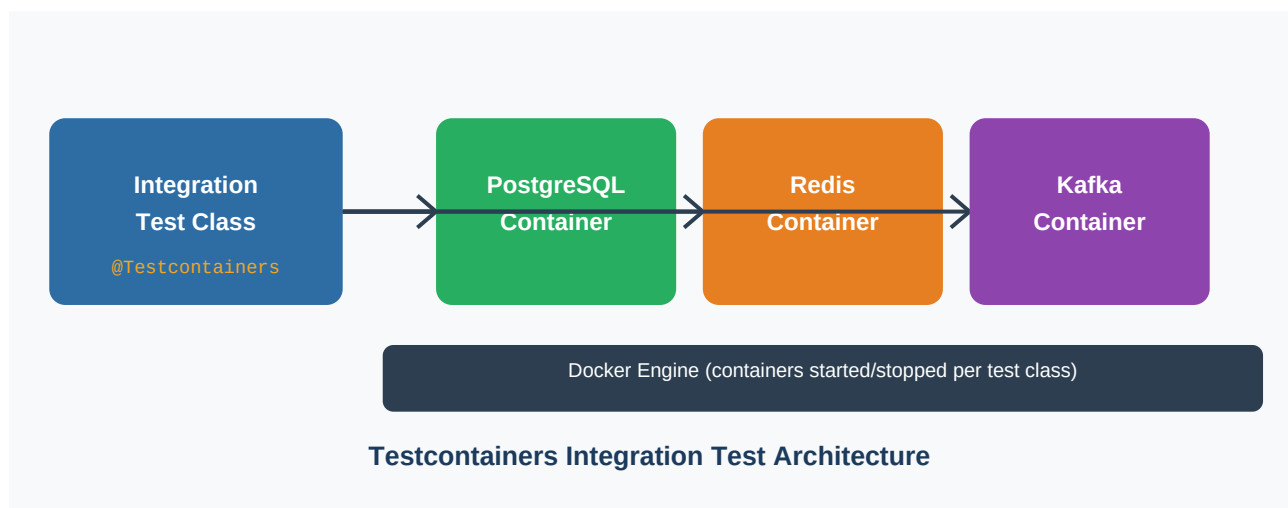
```
@DynamicPropertySource static void props(DynamicPropertyRegistry r) {
    r.add("spring.datasource.url", postgres::getJdbcUrl); r.add("spring.datasource.username",
    postgres::getUsername); r.add("spring.datasource.password", postgres::getPassword); }
@Autowired UserRepository repo; @Test void saveAndFind() { repo.save(new
    User("alice@example.com")); assertThat(repo.findByEmail("alice@example.com")).isPresent(); } }
```

Component Tests (Docker Compose)

- Deploy the service + all its dependencies via Docker Compose
- Test via HTTP (RestAssured or WebClient) — black-box testing
- No mocking — verifies service behavior with real DB, cache, and message broker
- Slower than integration tests; useful for full flow verification

```
@SpringBootTest(webEnvironment = RANDOM_PORT) @Testcontainers class OrderServiceComponentTest
{ @Container static DockerComposeContainer<?> env = new DockerComposeContainer<>(new
    File("docker-compose-test.yml")) .withExposedService("postgres", 5432)
    .withExposedService("redis", 6379); @Test void createOrder_returnsCreated() {
    RestAssured.given().contentType("application/json").body(orderPayload).post("/orders")
    .then().statusCode(201); } }
```

Testcontainers Architecture



Spring Boot Test Slices

Annotation	Loads	Use For
@WebMvcTest	Web layer only	Controller unit tests (MockMvc)
@DataJpaTest	JPA + in-memory DB	Repository/entity tests
@DataRedisTest	Redis layer	Redis repository tests
@JsonTest	JSON serialization	DTO serialization/deserialization
@SpringBootTest	Full context	Integration/component tests

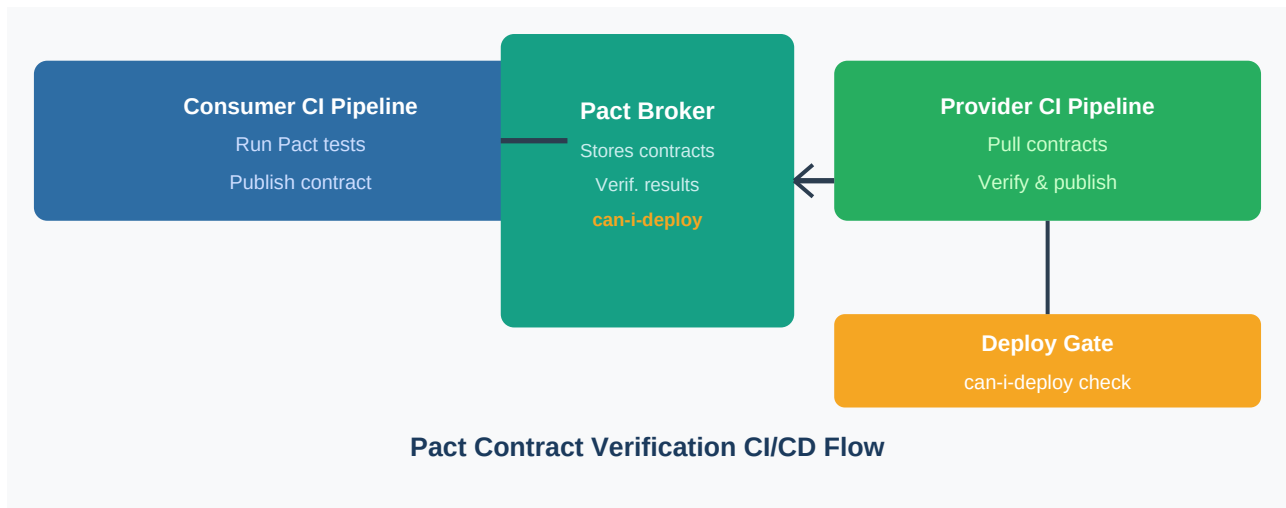
WireMock for External Service Stubs

- WireMock stubs HTTP calls to external services in integration tests
- @WireMockTest (spring-cloud-contract-wiremock) or standalone WireMockServer

- Record stubs from real services or define programmatically

```
@WireMockTest class PaymentClientTest { @Test void processPayment_success(WireMockRuntimeInfo
wm) { stubFor(post("/payments") .willReturn(aResponse() .withStatus(200)
.withHeader("Content-Type", "application/json") .withBody('{"status":"approved"}')));
PaymentResponse r = client.processPayment(new PaymentRequest(...));
assertThat(r.getStatus()).isEqualTo("approved"); } }
```

Contract Testing (Pact) CI Flow



Pact contract testing decouples consumer and provider testing. Consumers define what they need; providers prove they satisfy those needs in CI. The Pact Broker coordinates contracts and verification results, and the can-i-deploy command prevents breaking deployments.

E2E Test Strategy

- Minimal — cover critical happy paths only (checkout, login, core business flow)
- Run against full deployed environment (staging/pre-prod)
- Slow, brittle, expensive — do not duplicate unit/integration coverage
- Use REST Assured, Selenium, or Playwright for HTTP/UI flows
- Parallelize in CI to reduce total execution time

Test Data Management

- @Sql annotations to load test data before tests
- @Transactional on test class — rolls back after each test
- TestEntityManager for JPA test data setup
- FixtureFactory or object mothers for complex domain objects

```
@Test @Sql("/test-data/users.sql") // insert before test @Transactional // rollback after void
findActiveUsers_returnsNonEmpty() { List<User> users = repo.findAllActive();
assertThat(users).isNotEmpty(); }
```

Interview Q&A — Chapter 15.9: Microservices Testing Strategies

Q1. Describe the test pyramid and how it applies to microservices.

The test pyramid has broad base of fast unit tests, narrower middle of integration tests, and a small top of E2E tests. For microservices: Unit tests (mock dependencies, test business logic—many, milliseconds). Integration tests (Testcontainers, test single service with real DB/cache—fewer, seconds). Component tests (Docker Compose, test service via HTTP—few, seconds-to-minutes). Contract tests (Pact, verify inter-service compatibility—team-level). E2E tests (full flow, critical paths only—minimal, minutes).

Q2. What is Testcontainers and what problem does it solve?

Testcontainers is a Java library that starts real Docker containers (PostgreSQL, Redis, Kafka, etc.) during tests, providing authentic infrastructure instead of in-memory fakes. It solves the 'works in unit tests, fails in production' problem caused by behavioral differences between H2 in-memory DB and real PostgreSQL. Containers start before tests and are destroyed after, ensuring clean, isolated test environments.

Q3. How do you use @DynamicPropertySource with Testcontainers?

@DynamicPropertySource is a static method that registers property overrides after the container starts, injecting the container's dynamic port and connection details into the Spring application context. Example: `r.add('spring.datasource.url', postgres::getJdbcUrl)`. This works because @DynamicPropertySource runs before the Spring context initializes, allowing container-specific properties to be used by auto-configuration.

Q4. What is the difference between @Testcontainers with @Container static vs instance fields?

Static @Container: container is started once for the entire test class (class-scoped lifecycle). More efficient — avoids container restart between test methods. Instance @Container: container restarted for each test method — slower but complete isolation. Best practice: use static containers with @Transactional rollback for database tests to get both performance and isolation.

Q5. What are Spring Boot test slices and when would you use @WebMvcTest?

Test slices load only the portion of the Spring context needed for a specific test type. @WebMvcTest loads web layer (controllers, filters, security) but no service/repository beans. Use @MockBean for service dependencies. Use it to test: controller request/response mapping, validation annotations (@Valid), exception handlers, security configurations. Much faster than @SpringBootTest as it doesn't load the full application context.

Q6. How do you write a @WebMvcTest for a REST controller?

Annotate test with @WebMvcTest(UserController.class). Inject MockMvc via @Autowired. Mock service dependencies with @MockBean. Use `mockMvc.perform(get('/users/1')).andExpect(status().isOk()).andExpect(jsonPath("$.name").value('Alice'))`. Test validation by sending invalid requests and expecting 400. Test security by calling without auth and expecting 401.

Q7. What is the difference between integration tests and component tests for microservices?

Integration tests: test a single service with real infrastructure (DB, cache) via Testcontainers. Tests are at the code level — calling service/repository methods directly. Component tests: deploy the entire service (including app layer) + dependencies, test via HTTP as a black box — external behavior, not internal code paths. Component tests give more confidence about real behavior; integration tests are faster and more targeted.

Q8. What is WireMock and when should you use it?

WireMock is an HTTP mock server that stubs external HTTP service calls in tests. Use it when: testing a service that calls an external REST API that you don't control, simulating error scenarios (500, timeout, slow responses), testing retry/circuit breaker behavior. Unlike Pact, WireMock stubs are defined by the consumer without provider involvement. Spring Cloud Contract provides `spring-cloud-contract-wiremock` for easy Spring Boot integration.

Q9. How do you test retry and circuit breaker behavior?

Use WireMock to simulate failures: `stubFor(get('/service').willReturn(serverError()))` for first N calls, then `ok()` for subsequent calls. Verify circuit breaker opens after threshold failures. For Resilience4j: use `CircuitBreakerRegistry.of(config)` with low thresholds in test config. Use WireMock scenarios to model stateful behavior (FAILING state → RECOVERING state).

Q10. What is consumer-driven contract testing and how does Pact implement it?

Consumer-driven contract testing means the consumer defines what API interactions it needs. The provider must verify it satisfies those interactions. In Pact: consumer writes a test using `PactDslWithProvider` builder defining request/response. Pact generates a JSON contract file. Provider runs `@Provider` verification tests against the real application replaying each interaction. Both publish results to Pact Broker.

Q11. How do you write a Pact consumer test in JUnit 5?

Annotate with `@ExtendWith(PactConsumerTestExt.class)`. Create `@Pact` method returning `RequestResponsePact` using `PactDslWithProvider` builder: given state, upon receiving request, will respond with response. Write `@PactTestFor(pactMethod = 'createPact')` test method receiving the mock provider URL. Test your actual client code against the mock URL. Pact verifies the interaction and saves the contract.

Q12. What is the Pact Broker and what are its key features?

The Pact Broker is a central service storing contracts and verification results. Key features: stores contracts indexed by consumer+provider+version, tracks which provider versions have been verified against which consumer contracts, provides 'can-i-deploy' API/CLI checking deployment safety, supports tagging (production, staging, main branch), webhooks to trigger provider CI when new contracts are published.

Q13. What is 'can-i-deploy' in Pact and how does it prevent breaking changes?

'Can-i-deploy' queries the Pact Broker to check whether a specific version of a service is safe to deploy to a given environment. It checks: are all consumer contracts for this provider verified against this provider version? Are all provider contracts this consumer depends on verified? Integrated into CI/CD as a deployment gate: `'pact-broker can-i-deploy --participant user-service --version 1.2 --to prod'` returns non-zero exit if not safe.

Q14. What is the difference between contract tests and integration tests?

Integration tests require both services to be running and test real HTTP interactions — slow, environment-dependent, test coupling. Contract tests decouple testing: consumer tests run against a mock; provider tests verify contracts independently without the consumer running. Contract tests are fast (no inter-service HTTP calls in CI), run in isolation, and precisely identify which consumer's expectation a provider change breaks.

Q15. What are the challenges with E2E tests in microservices?

Fragility: any one of many services failing causes test failure (no isolation). Slowness: full service deployment and HTTP flows take minutes. Environment complexity: requires all services running with compatible versions. Test data management: state across services is hard to set up and tear down. Non-determinism: timing, retry, and async behavior cause flaky tests. Ownership: who owns cross-service E2E tests is often

unclear.

Q16. When should you write E2E tests vs relying on contract tests?

E2E tests for: critical business flows that would cause severe user impact (checkout, payment), flows spanning multiple async boundaries where contract tests can't catch timing issues, regulatory/compliance requirements. Rely on contract + component tests for: most inter-service interaction verification, error handling (much easier to simulate with WireMock/Pact), regression testing of individual services. Target: contract/component tests replace 90%+ of E2E scenarios.

Q17. What are @Sql and @Transactional used for in Spring integration tests?

@Sql('/test-data/users.sql') runs SQL scripts before (or after) a test method/class to set up data. @Transactional on a test class wraps each test in a transaction that is automatically rolled back after the test, leaving the database clean for the next test without needing explicit cleanup. Combine both: @Sql sets up data, @Transactional ensures rollback — tests are fast and independent of execution order.

Q18. What is the 'object mother' or 'test fixture' pattern?

Object mother is a factory class that creates pre-configured domain objects for tests. Example: UserMother.validUser(), UserMother.inactiveUser(), OrderMother.pendingOrder(). Benefits: centralizes test data creation, reduces duplication, makes tests more readable. For microservices: separate fixture module shared across test types (unit, integration, component) ensures consistent test data across layers.

Q19. How do you test Kafka producers and consumers with Testcontainers?

Use KafkaContainer from testcontainers-kafka. Configure Spring Kafka with container's bootstrap servers via @DynamicPropertySource. For producer tests: send message, consume with KafkaConsumer in test, verify message content. For consumer tests: use KafkaProducer to send test messages, wait with CountdownLatch or Awaitility for consumer to process, verify side effects. Use EmbeddedKafka (@EmbeddedKafka) for faster unit-level tests.

Q20. What is @DataJpaTest and what does it provide?

@DataJpaTest configures an in-memory database (H2 by default), scans @Entity classes and @Repository interfaces, configures JPA, but does not load @Service or @Controller beans. Provides TestEntityManager for test data setup. Each test is @Transactional by default. Limitation: uses H2 instead of real DB — behavior differences exist (case sensitivity, function support). Use @DataJpaTest with @AutoConfigureTestDatabase(replace=NONE) and Testcontainers for production-equivalent DB.

Q21. How do you test Spring Security in @WebMvcTest?

By default, @WebMvcTest includes security auto-configuration. Test unauthenticated: mockMvc.perform(get('/secured')).andExpect(status().isUnauthorized()). Test with mock user: @WithMockUser(roles = 'ADMIN') on test method. Test with JWT: use SecurityMockMvcRequestPostProcessors.jwt() to inject claims: .with(jwt().authorities('ROLE_USER')).jwt(j -> j.subject('user123')). Test @PreAuthorize by verifying 403 for insufficient roles.

Q22. What is mutation testing and when is it valuable in microservices?

Mutation testing (Pitest) introduces small code mutations (change != to ==, remove conditional) and checks if existing tests catch them. High mutation score indicates tests are effective. Valuable for: core business logic with high stakes (payment, inventory), identifying tests that pass but don't actually verify behavior. Run in CI for critical service modules. Slow — run incrementally on changed code only.

Q23. How do you test database migrations (Flyway/Liquibase) in CI?

Run migrations against a real DB via Testcontainers in CI. For Flyway: `@FlywayTest` or simply `@SpringBootTest` with Testcontainers — Flyway runs migrations on startup automatically. Test rollback: verify `V2__add_column.sql` can be rolled back by applying `V3__revert.sql`. Verify no data loss by seeding data before migration, migrating, verifying data intact. Flyway Enterprise supports undo migrations; open-source uses compensation migrations.

Q24. What is the 'test double' taxonomy in microservices testing?

Dummy: object passed but never used (null works). Fake: working implementation with shortcuts (in-memory list instead of DB). Stub: pre-programmed return values, no behavior verification (Mockito `when().thenReturn()`). Mock: records interactions, verifies them in assertions (Mockito `verify()`). Spy: partial mock of real object (Mockito `@Spy`). For microservices: stubs for predictable happy paths, mocks for verifying outgoing calls.

Q25. How do you handle flaky tests in a microservices CI pipeline?

Identify: track test history in CI, flag tests with >1% failure rate on passing code. Quarantine: move flaky tests to a separate non-blocking CI step. Fix categories: timing issues (use Awaitility instead of `Thread.sleep()`), shared state (ensure test isolation with Testcontainers), network timeouts (mock external calls), test ordering dependencies (enforce independent state). Track flakiness rate as a team metric.

Q26. What is Awaitility and when do you use it in microservices tests?

Awaitility is a DSL for waiting for asynchronous conditions without arbitrary sleeps. Example: `await().atMost(5, SECONDS).until(() -> messageRepo.findById(id).isPresent())`. Use in: event-driven tests waiting for Kafka message processing, async REST calls with `@Async` methods, cache warming tests, saga/workflow step completion. Avoids `Thread.sleep` flakiness — retries assertion until timeout.

Q27. What is 'shift-left' testing in microservices?

Shift-left means moving testing earlier in the development process. For microservices: write API contracts (Pact) before implementation, TDD for business logic, IDE plugins for instant feedback, pre-commit hooks running unit tests, fast feedback loops via test containerization. Shift-left reduces the cost of defects (cheap to fix in unit tests, expensive to find in E2E or production).

Q28. How do you test idempotency in microservices?

For HTTP: send the same request twice with the same idempotency key, verify second call returns same response without duplicate side effects. For Kafka consumers: publish the same event twice, verify exactly-once processing (check DB state, not message count). Test the idempotency store (Redis/DB) lookup: pre-seed with processed idempotency key, send request, verify it's rejected/returned from cache, no duplicate processing.

Q29. What is the 'strangler fig' testing strategy for migration?

When migrating from monolith to microservices: run both implementations simultaneously in tests (parallel run). Compare outputs byte-by-byte or semantically. Canary testing: route X% of production traffic to new service, compare results, gradually increase percentage. Regression test pack: capture production request/response pairs, replay against new service, verify equivalence.

Q30. How do you test saga patterns in microservices?

Choreography saga: publish events, verify each service's reaction in isolation (unit test with mocked event publishers, integration test with real Kafka). Orchestration saga: test the orchestrator with mocked step executions (verify compensation on failure). Integration test: use Testcontainers with all saga participants, inject failures at each step to verify compensation chains. Awaitility to wait for eventual saga completion.

Q31. What metrics should you track for test quality in a microservices pipeline?

Code coverage per service (target: 80%+ line coverage, 90%+ branch for core logic). Test execution time per layer (fail build if unit tests exceed 5 minutes). Flakiness rate (failed on passing code). Contract coverage (% of API surface covered by Pact contracts). Mutation score for critical modules. Time from test failure to detection in CI (test speed).

Q32. How do you test gRPC services in Spring Boot?

Use `grpc-spring-boot-starter-test` or `GrpcServerExtension` for JUnit 5. Start real gRPC server with `Testcontainers` or in-process server for unit tests. Test with generated gRPC stubs: `client.getUser(GetUserRequest.newBuilder().setId(1).build())`. For contract testing: use Pact's gRPC support or Spring Cloud Contract with protobuf stubs. Test error cases by mocking service to throw `Status.NOT_FOUND`.

Q33. What is the 'ice cream cone' anti-pattern in microservices testing?

The ice cream cone (inverted test pyramid) has few unit tests and many E2E tests — the opposite of the pyramid. Common in teams that migrated from monolith without updating test strategy. Symptoms: slow CI (>30 minutes), many flaky tests, long feedback cycles. Fix: identify E2E tests covering unit-testable logic, rewrite as unit/contract tests, reduce E2E to critical flows only.

Q34. How do you test event schema compatibility in Kafka?

Use Confluent Schema Registry with compatibility checks: BACKWARD, FORWARD, FULL. Test in CI: register current schema, attempt to register new schema, Schema Registry rejects incompatible changes. For Avro: use `AvroCompatibilityChecker` in tests. `Testcontainers` provides `SchemaRegistryContainer`. Test deserialization of old messages with new schema (BACKWARD compatibility).

Q35. What is TestRestTemplate vs WebClient?

`TestRestTemplate`: blocking, `RestTemplate`-based, for `@SpringBootTest` with `RANDOM_PORT`. Good for traditional synchronous REST testing. `WebClient`: reactive, fluent DSL, supports both sync and reactive applications. Better assertion API (`StepVerifier` for streaming, `expectBody` for JSON). Prefer `WebClient` for all new tests — supports both `WebMvc` and `WebFlux`. Example: `webClient.get().uri('/users/1').exchange().expectStatus().isOk().expectBody(UserDto.class)`.

Q36. How do you test rate limiting in a microservice?

Use a test profile with very low rate limits (e.g., 3 requests per second). Send requests in rapid succession (`TestRestTemplate` in loop or Gatling load test). Verify: first N requests return 200, subsequent return 429 Too Many Requests. Verify `Retry-After` header is present. Reset rate limiter between tests: call admin endpoint or use in-memory counter that resets. Test rate limit by user/API key isolation.

Q37. What is the role of 'test profiles' in Spring Boot microservices testing?

`@ActiveProfiles('test')` activates test-specific configuration. `application-test.yml` can override: use H2 instead of PostgreSQL (for unit tests), disable external service auto-configuration, set short timeouts, enable debug logging, use local Kafka broker address. Multiple profiles: `test` (unit), `integration-test` (`Testcontainers`), `component-test` (Docker Compose). Each profile gives appropriate trade-off between speed and realism.

Q38. How do you achieve test isolation in microservices integration tests?

Database: `@Transactional` rollback or `@Sql` to truncate tables before each test. Kafka: unique topic per test class or topic cleanup in `@BeforeEach`. Redis: `FLUSHDB` in `@BeforeEach` or use separate DB index per test. HTTP mocks: reset `WireMock` stubs in `@BeforeEach` (`wmServer.resetAll()`). Avoid shared mutable state. `Testcontainers` static containers provide shared infrastructure with per-test data isolation.

Q39. What is 'consumer-driven contract testing' vs 'provider-driven contract testing'?

Consumer-driven (Pact): consumer defines expected interactions; provider verifies them. Consumer has more power — changes must not break consumers. Provider-driven (OpenAPI/Spring Cloud Contract): provider defines the API spec; consumers must adapt to the spec. Provider has more control. Hybrid: use OpenAPI for documentation and schema validation, Pact for consumer-specific interaction verification. Consumer-driven is better for preventing unintentional breaking changes.

Q40. How do you test distributed tracing in microservices?

Unit tests: verify tracing headers are propagated in outgoing requests (check RestTemplate/WebClient interceptors pass traceparent header). Integration tests: verify trace ID appears in logs (capture log output, assert trace ID present). Component tests: use Zipkin/Jaeger Testcontainer, execute a request, query tracing backend to verify complete trace with all expected spans. Test that span attributes (service name, operation name) are correctly set.

Q41. What is the 'test pyramid' for Kafka-based microservices?

Unit: Kafka producer/consumer logic tested with mocked KafkaTemplate/ListenerContainerFactory. Integration: Testcontainers Kafka + Spring Kafka, real producer and consumer in same test. Component: service deployed, external test sends Kafka message, verifies HTTP state change. Contract: Pact for async messages (PactMessagingProvider) — consumer defines event schema, provider verifies it produces correct events. E2E: multi-service Kafka flow from event origin to final state.

Q42. How do you test authorization logic in microservices?

@WebMvcTest with Spring Security: test @PreAuthorize expressions by mocking Authentication. Use @WithMockUser(authorities = 'ROLE_ADMIN') for role-based tests. For JWT: MockMvcRequestPostProcessors.jwt() with custom claims. Test method security: call service method directly with SecurityContextHolder set. Verify 403 response for insufficient permissions, 401 for missing authentication. Test resource ownership checks (user can only access own resources).

Q43. What is TestNG vs JUnit 5 for microservices testing?

JUnit 5 (Jupiter): modern, extensible, @ExtendWith, parameterized tests, nested tests, better Spring Boot Test integration, wider ecosystem. TestNG: stronger support for test groups, parallel execution configuration, data providers. Recommendation: JUnit 5 for most microservices testing. JUnit 5 supports: @ParameterizedTest with @CsvSource for multiple scenarios, @Nested for organizing test contexts, @Timeout for preventing hung tests.

Q44. How do you handle test data in a shared Testcontainers DB?

Static container shared across test class. Use @Sql to insert data per test. @Transactional rollback ensures data doesn't persist between tests. For tests that can't use @Transactional (e.g., tests with multiple transactions): use @AfterEach cleanup: jdbcTemplate.execute('DELETE FROM orders WHERE test = true'). Tag test data with test-specific marker to enable safe cleanup without truncating tables.

Q45. What is Spring Cloud Contract and how does it differ from Pact?

Spring Cloud Contract (SCC): provider-driven — provider defines contracts in Groovy/YAML DSL. SCC generates provider tests and WireMock stubs for consumers. Consumers use generated WireMock stubs to test against. Pact: consumer-driven — consumer writes tests, generates contracts. Difference: Pact gives consumers control; SCC gives providers control. Pact requires Pact Broker; SCC contracts live in provider repo. Pact more widely adopted for polyglot environments.

Q46. How do you test observability (metrics/logs) in microservices?

Metrics: use Micrometer's SimpleMeterRegistry in tests, execute operations, assert Counter/Timer values. Example: `assertThat(meterRegistry.get('orders.created').counter().count()).isEqualTo(1)`. Logging: use capture log appender (logback ListAppender), execute operation, verify correct log messages at correct levels. Health checks: MockMvc GET /actuator/health, verify UP status and component details.

Q47. What is the 'test clock' pattern for time-dependent microservices tests?

Inject Clock as a Spring bean instead of using `Instant.now()` or `LocalDateTime.now()` directly. In tests, inject a fixed clock: `Clock.fixed(Instant.parse('2024-01-15T10:00:00Z'), UTC)`. Control time precisely: verify TTL expiry, scheduled job execution, date-based filtering, token expiry. In production: inject `Clock.systemUTC()`. This makes time-dependent behavior deterministic and controllable in tests.

Q48. How do you test microservice resilience (timeout, retry, fallback)?

Timeout: WireMock fixed delay: `willReturn(aResponse().withFixedDelay(5000))`. Verify service returns 503 or fallback response within configured timeout. Retry: WireMock scenarios — fail twice then succeed. Verify client retries and eventually succeeds. Circuit breaker: fail enough times to open circuit, verify subsequent fast-fail responses. Fallback: verify fallback method is called, returns degraded but functional response.

Q49. What is the recommended testing strategy for a microservice refactoring?

1. Capture characterization tests: record current behavior via contract tests/integration tests. 2. Refactor with TDD: unit tests for new structure. 3. Run characterization tests against new implementation — must pass. 4. Component tests verify HTTP behavior unchanged. 5. Pact contracts ensure no consumer-facing API changes. 6. Parallel run: both old and new code path, compare outputs. 7. Gradually shift traffic to new implementation.

Q50. How do you test microservice configuration in different environments?

`@TestPropertySource` to override properties in tests. `@SpringBootTest(properties = {"feature.flag=true"})` for one-off overrides. Test config loading from Vault/Config Server: mock config server with WireMock or use `@PropertySource` pointing to test-specific config. Test that required properties fail fast: verify `ApplicationContext` fails to start without required properties (`@ConfigurationProperties` with `@Validated`).

Q51. What are the performance testing considerations for microservices?

Load testing: Gatling or k6 scripts testing individual service endpoints under load. Chaos testing: Chaos Monkey for Spring Boot (`@EnableChaosMonkey`) introduces latency, exceptions, kills instances. Baseline performance tests in CI: measure p95/p99 latency, fail if exceeds threshold. Database connection pool sizing: test under concurrent load to detect pool exhaustion. Memory leak tests: run load for extended period, monitor heap growth.